

MatchPass Project

Versão do Documento: v0.0.3 | Especificação Completa de Entidades, DTOs (Records), Enums e Persistência

O **MatchPass** é uma plataforma focada na venda de ingressos para partidas de futebol, projetada sob os pilares de excelente experiência do usuário, escalabilidade linear e alta resiliência a falhas. O ecossistema adota uma arquitetura de microsserviços fracamente acoplados estruturados sob os conceitos de *Domain-Driven Design* (DDD). A comunicação transacional síncrona é efetuada via HTTP/REST através de um API Gateway, enquanto os fluxos de alta concorrência e consistência eventual utilizam o **RabbitMQ**.

1. Estratégia de Persistência Poliglota (Database-per-Service)

Para assegurar total isolamento e autonomia, cada microsserviço gerencia e possui acesso exclusivo à sua própria base de dados de acordo com os requisitos de desempenho e modelo de dados do domínio:

Microsserviço	Tipo de Banco	Tecnologia	Justificativa Técnica
Event-Catalog-Service	Relacional	PostgreSQL	Dados estruturados com relacionamentos consistentes entre estádios, setores e partidas.
Inventory Service	Em Memória / NoSQL	Redis	Alta velocidade de leitura e escrita para gerenciar travas de assentos (Locks) com TTL automático durante picos de concorrência.
Checkout & Order Service	Relacional	PostgreSQL	Exige consistência estrita (ACID) para manipulação segura e auditoria do histórico de pedidos dos clientes.
Payment Service	Relacional	PostgreSQL	Garantia transacional rígida sobre logs financeiros, integrações externas e reconciliação de transações.
Ticket & Access Service	Relacional	PostgreSQL	Armazenamento seguro de ingressos emitidos e logs relacionais rígidos para validação de entrada nas catracas físicas.
Identity Service	Relacional	PostgreSQL	Garantia de integridade referencial rigorosa para credenciais de acesso, dados do usuário e permissões.
Notification Service	NoSQL de Documentos	MongoDB	Esquema dinâmico para persistir diferentes payloads de histórico de envios e alto volume de logs de forma barata.

2. Modelagem Detalhada dos Microserviços

Esta seção descreve a implementação técnica exata de cada componente do sistema, padronizada com Java moderno (Records para DTOs, Enums fortificados e anotações específicas de persistência).



Event-Catalog-Service (Core Domain)

Responsável pelo ciclo de vida dos eventos esportivos, times e estádios. Serve como a origem dos dados estruturais.

```
// Entidades JPA (PostgreSQL)
@Entity
public class Venue {
    @Id private UUID id;
    private String name;
    private String city;
    private String state;
    private Integer totalCapacity;
    @OneToMany(mappedBy = "venue", cascade = CascadeType.ALL)
    private List<Sector> sectors;
}

@Entity
public class Sector {
    @Id private UUID id;
    private String name;
    private Integer capacity;
    private Boolean hasNumberedSeats;
}

@Entity
public class Event {
    @Id private UUID id;
    private String title;
    private LocalDateTime eventDate;
    @Enumerated(EnumType.STRING) private EventStatus status;
    private UUID venueId;
    private UUID homeTeamId;
    private UUID awayTeamId;
    @OneToMany(mappedBy = "event", cascade = CascadeType.ALL)
    private List<EventSectorPricing> pricings;
}

@Entity
public class EventSectorPricing {
    @Id private UUID id;
    private UUID sectorId;
    private BigDecimal basePrice;
    private BigDecimal halfPrice;
    private Boolean isAvailable;
}

// Enums
public enum EventStatus { SCHEDULED, CANCELED, FINISHED }

// DTOs (Records)
public record CreateEventRequest(String title, LocalDateTime eventDate, UUID venueId, UUID
homeTeamId, UUID awayTeamId, List<SectorPricingRequest> sectorsPricing) {}
public record SectorPricingRequest(UUID sectorId, BigDecimal basePrice, BigDecimal
halfPrice) {}
public record EventSummaryResponse(UUID eventId, String title, LocalDateTime eventDate,
String venueName, String venueCity, BigDecimal startingPrice) {}
public record EventDetailsResponse(UUID eventId, String title, LocalDateTime eventDate,
VenueDto venue, List<SectorPricingResponse> availableSectors) {}
```

```
public record SectorPricingResponse(UUID sectorId, String sectorName, BigDecimal basePrice,
Boolean hasNumberedSeats) {}
```

Inventory Service (Core Domain)

Camada reativa de alta concorrência encarregada de controlar o estado em tempo real das cadeiras.

```
// Entidade em Memória (Redis Hash)
@RedisHash("SeatLock")
public class SeatLock {
    @Id private String lockId;
    @Indexed private UUID eventId;
    @Indexed private UUID sectorId;
    private UUID userId;
    private Integer quantity;
    private String seatNumber;
    private SeatStatus status;
    @TimeToLive private Long ttl; // Tempo de expiração automática (ex: 600 segundos)
}

// Enums
public enum SeatStatus { AVAILABLE, LOCKED, SOLD }

// DTOs (Records)
public record SeatReservationRequest(UUID eventId, UUID sectorId, Integer quantity, String
seatNumber) {}
public record SeatStatusResponse(String lockId, SeatStatus status, LocalDateTime expiresAt)
{}
}
```



Checkout & Order Service (Core Domain)

Orquestra a criação do carrinho de compras e gerencia o estado transacional dos pedidos.

```
// Entidades JPA (PostgreSQL)
@Entity
@Table(name = "orders")
public class Order {
    @Id private UUID id;
    private UUID userId;
    private UUID eventId;
    private String seatLockId;
    private BigDecimal totalAmount;
    @Enumerated(EnumType.STRING) private OrderStatus status;
    private LocalDateTime createdAt;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
    private List<OrderItem> items;
}

@Entity
public class OrderItem {
    @Id private UUID id;
    private UUID sectorId;
    @Enumerated(EnumType.STRING) private TicketType ticketType;
    private BigDecimal appliedPrice;
}

// Enums
public enum OrderStatus { PENDING, AWAITING_PAYMENT, COMPLETED, CANCELED, REFUNDED }
public enum TicketType { INTEIRA, MEIA_ESTUDANTE, MEIA_SOLIDARIA }

// DTOs (Records)
public record CheckoutRequest(UUID eventId, List<OrderItemRequest> items) {}
public record OrderItemRequest(UUID sectorId, TicketType ticketType, String seatNumber) {}
public record OrderSummaryResponse(UUID orderId, BigDecimal totalAmount, OrderStatus
status, String paymentUrl) {}
```

Payment Service (Generic Subdomain)

Responsável pelo desacoplamento financeiro e integração técnica com gateways externos.

```
// Entidades JPA (PostgreSQL)
@Entity
public class Transaction {
    @Id private UUID id;
    private UUID orderId;
    private UUID userId;
    private BigDecimal amount;
    @Enumerated(EnumType.STRING) private PaymentProvider provider;
    private String providerTransactionId;
    @Enumerated(EnumType.STRING) private TransactionStatus status;
    private LocalDateTime processedAt;
}

// Enums
public enum TransactionStatus { PENDING, APPROVED, REFUSED, CHARGEBACK, ERROR }
public enum PaymentProvider { STRIPE, MERCADO_PAGO, PIX }

// DTOs (Records)
public record PaymentIntentRequest(UUID orderId, PaymentProvider provider, String
paymentMethodToken) {}
public record PaymentResultResponse(UUID transactionId, TransactionStatus status, String
qrCodePix) {}
```

Ticket & Access Service (Supporting Subdomain)

Camada que gerencia os bilhetes digitais e autentica a entrada nas catracas físicas do estádio.

```
// Entidades JPA (PostgreSQL)
@Entity
public class Ticket {
    @Id private UUID id;
    private UUID orderId;
    private UUID eventId;
    private UUID userId;
    private UUID sectorId;
    private String seatNumber;
    private String qrCodeHash; // Assinatura temporal dinâmica anti-cambismo
    @Enumerated(EnumType.STRING) private TicketStatus status;
}

@Entity
public class AccessLog {
    @Id private UUID id;
    private UUID ticketId;
    private String gateId;
    private LocalDateTime accessedAt;
    @Enumerated(EnumType.STRING) private AccessResult result;
}

// Enums
public enum TicketStatus { VALID, USED, REVOKED }
public enum AccessResult { GRANTED, DENIED_USED, DENIED_REVOKED, DENIED_INVALID }

// DTOs (Records)
public record ValidateAccessRequest(String qrCodeHash, String gateId) {}
public record AccessValidationResponse(Boolean isAllowed, AccessResult result, String
message, String sectorName, String seatNumber) {}
```

Identity Service (Generic Subdomain)

Centralizador de governança de segurança, controle de perfis e emissão de tokens JWT.

```
// Entidades JPA (PostgreSQL)
@Entity
@Table(name = "users")
public class User {
    @Id private UUID id;
    private String fullName;
    @Column(unique = true) private String email;
    private String passwordHash;
    @Column(unique = true) private String document; // CPF
    @Enumerated(EnumType.STRING) private UserRole role;
    private Boolean isActive;
}

// Enums
public enum UserRole { CUSTOMER, ADMIN, GATE_OPERATOR }

// DTOs (Records)
public record UserRegistrationRequest(String fullName, String email, String password,
String document) {}
public record AuthRequest(String email, String password) {}
public record AuthTokenResponse(String accessToken, String refreshToken, Long expiresIn) {}
```

Notification Service (Generic Subdomain)

Consumidor de mensagens puramente assíncrono para envios transacionais. Não expõe endpoints HTTP.

```
// Documento NoSQL (MongoDB)
@Document(collection = "notification_logs")
public class NotificationLog {
    @Id private String id;
    private UUID userId;
    private String recipientAddress;
    @Enumerated(EnumType.STRING) private NotificationType type;
    private String subject;
    private String payload;
    @Enumerated(EnumType.STRING) private NotificationStatus status;
    private String errorMessage;
    private LocalDateTime sentAt;
}

// Enums
public enum NotificationType { EMAIL, SMS, PUSH }
public enum NotificationStatus { SENT, FAILED }
```

3. O Fluxo Crítico de Compra via RabbitMQ (Padrão Saga)

Para contornar o problema de concorrência massiva e garantir que nenhum assento seja vendido duas vezes, aplica-se a coreografia de eventos assíncronos:

Passo 1: O cliente inicia o checkout. O **Checkout Service** cria um pedido com status PENDING e publica a mensagem `OrderPlacedEvent` no RabbitMQ.

Passo 2: O **Inventory Service** consome a mensagem e valida a disponibilidade no Redis. Se livre, insere o `SeatLock` temporário e publica `SeatReservedEvent`. Se ocupado, publica `SeatReservationFailedEvent`.

Passo 3: O **Payment Service** reage estritamente ao `SeatReservedEvent` e processa a transação com a operadora de crédito.

Passo 4: Caso a transação falhe, o evento `PaymentRefusedEvent` é gerado. O Inventário libera a cadeira instantaneamente no Redis, e o Checkout invalida o pedido.

Passo 5: Se o pagamento for bem-sucedido, o evento `PaymentApprovedEvent` é disparado. O Checkout altera o status para COMPLETED, o **Ticket Service** gera o QR Code e o **Notification Service** envia o e-mail final ao torcedor.

4. Guia de Infraestrutura: Bibliotecas Compartilhadas (Shared Libraries)

A biblioteca compartilhada (ex: `matchpass-common`) é um projeto de software isolado cujo objetivo é centralizar os contratos de mensagens que trafegam pelo Message Broker, gerando um artefato binário reutilizável (arquivo `.jar`) importado nos arquivos de configuração (Maven/Gradle) dos microserviços.

Contrato de Mensageria do Catálogo (na Biblioteca Compartilhada):

```
public record EventCreatedEvent(  
    UUID eventId,  
    UUID venueId,  
    LocalDateTime eventDate,  
    List<SectorCapacityInfo> sectors  
) {}  
  
public record SectorCapacityInfo(  
    UUID sectorId,  
    Integer capacity,  
    Boolean hasNumberedSeats  
) {}
```

Aviso Arquitetural contra Monolito Distribuído

Esta biblioteca comum deve permanecer extremamente enxuta (*thin library*). Ela só deve conter DTOs de eventos, Enums de estado globais e exceções base de infraestrutura. **Nunca insira regras de negócio, lógicas de validação ou entidades de banco de dados (JPA/Hibernate) dentro dela**, mantendo estrita a independência de implantação de cada microsserviço.